

RESEARCH ARTICLE

Real-Time TCP Packet Loss Prediction Using Machine Learning

MICHAEL WELZL¹, (Member, IEEE), SAFIQUUL ISLAM², (Senior Member, IEEE),
AND MAXIMILIAN VON STEPHANIDES³

¹Department of Informatics, University of Oslo, 0316 Oslo, Norway

²Department of Computer Science, Oslo Metropolitan University, 0130 Oslo, Norway

³Dintero AS, 0484 Oslo, Norway

Corresponding author: Michael Welzl (michawe@ifi.uio.no)

ABSTRACT Congestion and resulting packet loss in TCP connections can lead to performance degradation and reduce the Quality of Experience (QoE) for end users. Many common TCP congestion control algorithms therefore adjust their sending rate proactively, using heuristics based on measured delay and/or throughput. We have investigated whether it would be possible to replace such heuristics with Machine Learning (ML), such that Internet hosts could meaningfully react when a packet loss is predicted to happen. For example, such a reaction could involve the use of Forward Erasure Correction (FEC) to protect against the loss, or to dynamically tune parameters of TCP. We present two ML models that were trained for Reno and Cubic flows, respectively, and show that they can be beneficially applied in real time. We do this by informing a TCP sender to reduce its rate via a local, artificially generated Explicit Congestion Notification (ECN) signal at the sender before packet loss would have happened.

INDEX TERMS TCP, congestion control, machine learning, ECN.

I. INTRODUCTION

Traditional congestion control algorithms such as Reno [1] and Cubic [2] adjust the sending rate *reactively* when they see a congestion signal in the form of packet loss. This leads to a behavior where the sender keeps increasing its rate until congestion occurs, eventually causing packets to be dropped. Congestion and resulting packet loss can lead to performance degradation and reduced Quality of Experience (QoE) for end users.

One way to circumvent these detrimental effects is to react *proactively* and avoid packet loss. By allowing the sender to reduce its rate *before* congestion occurs, packet loss and resulting retransmissions are reduced, which reduces latency and improves the QoE for end users. This can be done by monitoring throughput or delay (as seen via returning acknowledgments (ACKs))—a strategy that is incorporated in many modern congestion control mechanisms, such as BBR [3], Low Extra Delay Background Transport (LEDBAT) [4], Vegas [5], COPA [6],

Network-Assisted Dynamic Adaptation (NADA) [7] and Self-Clocked Rate Adaptation for Multimedia (SCReAM) [8], to name but a few. It can also be done with Explicit Congestion Notification (ECN) [9], where routers set a bit in the IP header upon impending congestion, and the receiver uses another bit in the TCP header to tell the sender that it should back off.

With this paper, we do not participate in the competition for better congestion control, nor do we take a side in it regarding the choice of input heuristic. Instead, we try to answer the following question: *Since heuristics based on sender-side monitored delay and throughput metrics can be used to determine when a sender should back off, could we also use ML on these metrics to predict, in real time, when a packet loss would happen?* Such a prediction could be used in various ways, e.g. as input for a new congestion control mechanism, or by amending an existing one, e.g. to increase more cautiously or back off early; adding Forward Erasure Correction (FEC) to the data [10], but only at the right moment, to minimize unnecessary overhead; deciding whether starting a new connection should be slightly delayed; or adapting parameters of TCP pacing [11].

The associate editor coordinating the review of this manuscript and approving it for publication was Wai-Keung Fung¹.

We opted for a simple and protocol-independent approach to make use of our predictions, using ECN. An ECN bit, indicating congestion, is normally set by a router. Instead, when our prediction system identifies that packet loss would happen, we set the ECN bit directly at the sender, telling it to back off. Note that we *only* use ECN in this fashion; predicting whether an Active Queue Management (AQM) algorithm [12] or a scheme such as L4S [13] would ECN-mark packets would be interesting research in its own right, but this is beyond the scope of this paper.

A. CONTRIBUTIONS

We have done extensive research on the topic of real-time packet loss prediction, including the choice of a suitable ML model and informative TCP state variables that are readily available at a sender, where they could be used to select or construct machine learning features. The following bullet points summarize our contributions, which we will present after a brief introduction to Reno and Cubic congestion control and a discussion of related work in the following section:

- We have trained and tuned two XGBoost [14] models, one for Reno and one for Cubic, using data collected from numerous emulation tests with Reno and Cubic connections over a single bottleneck (Section III). While such tests cannot capture the true variability of real networks, they serve as a starting point to understand the feasibility of the idea.
- We elucidate our input metric selection and feature design process by explaining why we chose these metrics/features and what weights our models assigned to the features (Section III).
- We show results from real-time model inference tests, where we use ECN to make a TCP Reno or Cubic sender back off before loss would have happened. This clarifies that i) our models can indeed be applied in a practical system in real time, ii) these models can yield a benefit, even when using a very simple usage strategy (Section IV).

Why Reno and Cubic? We have chosen the Reno and Cubic congestion control mechanisms because they are both strictly loss based; Reno is a well known scheme with significant historic value, while Cubic is the most used TCP variant in the Internet [15]; both are standardized by the Internet Engineering Task Force (IETF) in RFCs with a “Proposed Standard” designation [1], [2].

We do not consider the second most used TCP variant, BBR [3] because it is not *only* loss-based, and hence, making it back off via ECN due to our predictions may yield inefficient and complex interactions. It is also a moving target: the first version was criticized for being too aggressive in competition with Cubic [16]; this was improved in version 2. Version 3, the most recent version at the time of writing, has again been criticized for being too aggressive against Cubic [17]. A measurement study shows that its use has been

declining [18]. BBR is not published as a standard, and the ongoing effort to standardize it aims at an “Experimental” designation [19].

II. BACKGROUND AND RELATED WORK

Since they are core elements of our investigation, we begin this section with a very brief introduction to the Reno and Cubic congestion control mechanisms in TCP. Then, we will discuss related work: ML applications for congestion control and network performance metric prediction.

A. BACKGROUND: RENO AND CUBIC

Both Reno and Cubic are strictly loss-based congestion control mechanisms, which means that they increase the congestion window (cwnd)—a variable that limits how many bytes can be “in flight” in the network at a given time—until packet loss happens (the idea being that this indicates congestion because a queue has overflowed). Alternatively, an ECN notification may arrive from the receiver; in accordance with the standard, both Reno and Cubic react to an ECN signal exactly as if a packet had been dropped [9].

Upon loss or an ECN notification, they reduce the cwnd by multiplying it with a factor β (Reno: $\beta = 0.5$, Cubic: $\beta = 0.7$). Generally in TCP, packet loss is indicated by the arrival of at least three duplicate acknowledgements (“DupACKs”); these are caused by out-of-order packet arrivals at the receiver. A transfer begins with a short “Slow Start” phase in which the cwnd increase is exponential. In case of very heavy congestion (e.g., all packets being lost), a retransmission timeout (RTO) will fire, causing TCP to enter the Slow Start phase again.

Long lasting connections in “normal” conditions, such as the ones we study here, primarily operate in a phase called “Congestion Avoidance” (CA), and the increase behavior in this phase constitutes the most significant difference between Reno and Cubic:

Reno: Every Round-Trip Time (RTT), cwnd is increased by one packet’s worth of data. Combined with the multiplicative decrease upon packet loss, this yields an “Additive Increase, Multiplicative Decrease” (AIMD) function resembling a saw-tooth. The long-term average cwnd of Reno, AVG_W_{Reno} , is commonly approximated using the following equation [20]:

$$AVG_W_{Reno} = \sqrt{\frac{8}{3p}} \quad (1)$$

with p being the loss event rate, i.e. one or more packet drops within an RTT; note that this is approximately the same as the packet loss ratio when p is small, which is a prerequisite for its long-term operation in CA anyway.

We note that it is also common to refer to Reno congestion control as “NewReno”, but this distinction is a mistake: “NewReno” really only refers to an improvement of how the original Fast Recovery algorithm interprets ACKs [21].

Cubic: Whenever an ACK arrives, the sender carries out a series of calculations. First, it determines:

$$W_{\text{cubic}}(t) = C \left(t - \sqrt[3]{\frac{W_{\text{max}} - \text{cwnd}_{\text{epoch}}}{C}} \right)^3 + W_{\text{max}} \quad (2)$$

where C is a constant (recommended as $C = 0.4$), t is the elapsed time in seconds in the current CA “stage” (i.e., since the beginning of the increase), W_{max} is the value of cwnd just before its last reduction, and $\text{cwnd}_{\text{epoch}}$ is the value of cwnd at the beginning of the current CA stage.

$W_{\text{cubic}}(t + \text{RTT})$ is then used as a target, which is lower-bounded with cwnd to ensure that it never causes a decrease, and upper-bounded with $1.5 \times \text{cwnd}$ to avoid being faster than Slow Start. Finally, cwnd is incremented by $\frac{\text{target} - \text{cwnd}}{\text{cwnd}}$, which yields a concave increase behavior when $\text{cwnd} < W_{\text{max}}$ and a convex increase behavior when $\text{cwnd} \geq W_{\text{max}}$ (Cubic is said to be in the “concave region” or in the “convex region”). The latter, more aggressive increase is justified by the fact that exceeding W_{max} indicates that network conditions have changed—e.g., other flows may have terminated.

Cubic was designed for network paths that have a large bandwidth $\times$ delay product (BDP). When the BDP is small, the above logic can sometimes yield a more conservative outcome than Reno. To prevent this, Cubic also estimates the cwnd of a sender implementing an AIMD-like behavior and uses it as a lower bound for $W_{\text{cubic}}(t)$. Cubic is said to be in the “Reno-friendly region” when this limit applies.

With the common choices of $C = 0.4$ and $\beta = 0.7$, the long-term average cwnd of Cubic, $\text{AVG_}W_{\text{Cubic}}$, can be approximated as [2]:

$$\text{AVG_}W_{\text{Cubic}} = 1.054 \frac{\sqrt[4]{\text{RTT}^3}}{\sqrt[4]{p^3}} \quad (3)$$

where p is the packet loss ratio (specifically, the number of packets between two successive packet losses is $\frac{1}{p}$).

B. RELATED WORK

Using ML to improve TCP is not a novel concept; following the seminal work on Remy [22], [23], many studies have investigated how ML can be applied to either develop new or improve existing congestion control mechanisms [24]. Some early examples are PCC [25] and Vivace [26]. Later, Deep Reinforcement Learning (DRL) has become the prevalent design approach for ML based congestion control. A vast amount of RL-based congestion control mechanisms have been brought forward—e.g., Indigo [27], Aurora [28], R3Net [29], MVFST-RL [30], Orca [31], QTCP [32], TCP-Drinc [33], SmartCC [34], Owl [35] and Astraea [36].

Strictly speaking, the application of our ML models to congestion control that we will present in Section IV yields a change to TCP that could also be categorized as an ML-based congestion control algorithm. However, besides this broad categorization, there are no similarities. All of the above

mechanisms auto-construct or tune congestion control behavior, which is not the goal of our ML models. Instead, we train TCP packet loss prediction models, and show that these models can be applied in real time—a topic that, to the best of our knowledge, has not been described before. The material in this paper therefore presents the first work on TCP packet loss prediction for real-time inference, and should be regarded as an initial proof of concept investigating its feasibility. The models that we present add a signal to the list of available signals that congestion controls usually work on; it can be used to make an earlier back-off decision (as we will investigate), or for completely other means—e.g., to dynamically enable FEC, or make decisions about when a new connection should be started. In the following, we give an overview of other work that predicts network performance metrics.

NETWORK PERFORMANCE METRIC PREDICTION

L. Roychoudhuri and E. S. Al-Shaer [37] propose a novel framework for predicting packet loss and congestion in real-time audio streams, based on end-to-end delay variation and trends, enabling proactive error recovery and congestion avoidance. They designate the minimum delay of a path as the baseline delay, signifying the delay under no congestion. They then identify the delay at the capacity saturation point of a path as the loss threshold delay, after which packet loss is more likely, and use the increase patterns or trends of the delay as an indication of congestion causing packet loss. The described application of the framework is similar to what we investigate: a form of proactive congestion avoidance based on real-time packet loss prediction. Doing this with heuristics instead of ML, however, puts it in the same group as all delay-based congestion detectors that are embedded in proactive congestion control mechanisms.

A paper by Nunes et al. [38] is one of the earlier works that used ML algorithms to predict network performance. It presents a novel approach to RTT estimation. Their findings indicated that their ML based RTT estimation is more accurate than the Exponentially Weighted Moving Average (EWMA) estimation that is used in most TCP congestion control mechanisms. The authors document experiments showing a reduction in the number of retransmitted packets and an increase in goodput compared to the traditional approach using an EWMA. TCP uses RTT estimates to compute its retransmission timeout (RTO) timer value; more accurate RTT estimations can therefore result in more accurate RTO values and less false retransmissions due to RTO expirations when the RTO should have been higher.

Giannakou et al. [39] discuss an ML tool based on Random Forest regression for predicting packet retransmissions in large scientific data transfers (e.g., from physics experiments). They predict the number of retransmissions using offline regression, not packet loss or congestion, and not proactively in real time, and therefore they do not deal with the exact problem that we are tackling. Nevertheless, the fact that a tree-based algorithm is used to construct an ML model based on various features such as cwnd and rtt makes this

work closely related to ours. Some other papers [40], [41] are quite similar to the paper discussed above in the sense that the focus is either on predicting the *amount* or *rate* of packet loss, but not if packet loss will occur at any given point in time.

Several works deal with predicting “congestion” in general [42], [43], [44], [45], in the network as a whole or in specific links in the network, sometimes hours or minutes beforehand. This is a much larger time-scale than in our case, as we deal with real-time prediction of loss and resulting actions in the order of round-trip times.

The authors of [46] discuss loss ratio prediction in IoT networks with time series data and Deep Learning (DL) models. Like some of the studies discussed above, this is not directly related to our work, which deals with actual packet loss prediction in real-time (a binary value), not a packet loss *rate* or *ratio*. The former is binary and the latter is a number, and it is quite a different (and inevitably time-averaged) metric. A proactive congestion control solution based on packet loss rate prediction is mentioned as future work.

The deep-learning based TCP (DL-TCP) protocol for a disaster 5G mmWave network uses indicators such as a node’s mobility information, location, signal strength and signal-to-noise ratio [47]. It adjusts a buffer and the TCP cwnd by predicting when the network is dis- or re-connected, leading to better network stability and higher throughput than with existing protocols such as TCP Reno, TCP Cubic, and TCP BBR.

5G mmWave congestion prediction is also the topic of a paper by Diez et al. [48]. They identify transport-layer metrics relevant to the congestion state, such as delay and inter-arrival time, and study their correlation with the perceived congestion. This is done by generating transport-layer traces and analyzing the information provided therein to derive meaningful congestion metrics. The paper finds a clear correlation between the moving standard deviation of the delay and congestion, but only a weak correlation between the moving average of the delay and congestion. These findings could be informative for an ML feature selection and construction process such as ours, although 5G mmWave connections are not a part of our considered dataset.

Different from what we present, none of these prior works focus on packet loss prediction at timescales that are relevant for TCP in real-time. Importantly, this is not only a limitation on inference speed: it is an entirely different matter to, e.g., predict a packet loss ratio or “congestion” as a statistical measure over a given time interval (as in the works above) than to predict singular packet losses that influence TCP, at TCP operation time scales (as we do). Our predictions can be used to meaningfully adapt TCP’s behavior, or they can be applied in other online network algorithms (e.g., to make decisions about when to start new connections, tune TCP parameters, or add FEC as a means to cope with packet loss).

III. MACHINE LEARNING MODEL DESIGN AND EVALUATION

In the following, we will explain what type of model we chose and why, and discuss how we obtained training data

and selected features. Then we will give a description of our training process.

A. MODEL CHOICE

A TCP connection can be interpreted as a time-series of TCP states, where the state of the connection at any given point (in the form of state variables such as `cwnd` and `rtt`) represents the probability of congestion. Congestion in the form of packet loss is a binary condition; we therefore phrase our problem as a binary classification, where we try to predict whether packet loss will happen or not. We chose to model our data with the widely used XGBoost [14] model with the “binary:logistic” loss function (logistic regression). We selected XGBoost because research has shown that it is among the top choices for tabular data [49]. XGBoost can even outperform Long Short Term Memory (LSTM) sequence models for time based prediction problems [50], [51]. We will now explain how we obtained suitable tabular data as input for this model.

B. OBTAINING TRAINING DATA AND FEATURE SELECTION

1) DATA GENERATION

We obtained the features by running emulated iPerf TCP tests between a sender and a receiver across a single bottleneck in realistic scenarios using mininet¹ network emulation. All tests ran for 300 seconds (5 minutes). As network configuration parameters, we changed the capacity from 10-50 Mbps in steps of 10, link delay from 30-70 ms in steps of 10, and used a queue length of a quarter, half and the full bandwidth×delay product (BDP) for each of these cases. In all of these network configurations, we monitored one single TCP connection in four separate scenarios: i) alone, i) with 6 Reno flows, or ii) with 6 Cubic flows, and iii) with 3 Reno and 3 Cubic flows. We used 6 flows as background traffic because this is a common upper limit for the number of connections that browsers open to the same DNS name, even with HTTP/2 [52], [53], giving us a typical competition scenario that Internet users may experience. Real Internet traffic is more complex, and the variability of traffic can be greater—hence, any results obtained with these data can only serve as an intermediate step towards a real-life system, for which additional training with actual Internet data would be recommendable.

2) DATA COLLECTION

For the monitored connection, we tracked the sender side variables with `ss`.² Referring to the `ss` man page,³ there are many fields that are more or less useful (informative) for our model. Most of the time, the output from `ss` contained all the

¹<http://mininet.org/>

²`ss` is a Linux network utility for dumping socket statistics. `ss` can be used to display internal TCP state information like the congestion window (`cwnd`), Slow Start threshold (`ssthresh`), and the Round-Trip Time (`rtt`). It is possible to supply `ss` with filter options such as source and destination IP address and port. This makes it possible to, for example, only capture TCP information matching a specific IP address and port pair.

³<https://man7.org/linux/man-pages/man8/ss.8.html>

fields that we deemed suitable as machine learning features in the training data, but there were cases where some of the fields were missing from the `ss` output. We discarded these cases. Note that missing data is not only a problem of `ss`, but it can be a more general issue when tracing variables of TCP at high speeds (for example, it was also a known problem of “Web10G”). Training our model with data that has some missing values due to processing speed limits ensures that it can work with this realistic limitation.

3) FEATURE SELECTION

Table 1 presents a comprehensive overview of the chosen features along with a short description for each feature. It seems obvious that the `cwnd` could be a useful feature in addition to the `rtt` because both the `cwnd` and `rtt` grow as packet loss becomes more and more likely. A large value for one of these metrics these could therefore indicate congestion, telling our model that packet loss could happen in the near future. We also crafted `min_cwnd` and `max_cwnd` features from the `ss` output because a large `cwnd` value may not be meaningful on its own; instead, a large `cwnd` value that is quite close to the `max_cwnd` seen so far most likely indicates a high chance of congestion, especially in cases when the `max_cwnd` does not fluctuate much (note that Cubic also uses the previous `max_cwnd` in its congestion increase function). Similarly, a `cwnd` value close to `min_cwnd` most likely indicates a small chance of congestion, because congestion has probably just been experienced and reacted upon. Generally, the `min_x` and `max_x` features serve as a way to incorporate historical information into the algorithm. By doing so, theoretically, as a connection progresses, the algorithm’s predictions could improve by learning the connection’s behavior (although, note, that we apply offline, not online learning, i.e. in fact it is the data quality that improves as a connection progresses).

We additionally created a feature called `cwnd_diff` which, as the name suggests, encodes the difference between the current and previous `cwnd` value. We considered this difference important because, without it, the model has no way to know if `cwnd` has been growing (and: how quickly) or declining.

4) DATA LABELING

Our goal is to detect the event where TCP should (or would, unless we prevent it) react to packet loss.⁴ We are thus primarily interested in the first occurrence of a loss within what is called “loss event” in TCP (one or more losses within the same window, or RTT). At first, we tried to identify this event with a simple logic using our `cwnd_diff` feature: if `cwnd_diff` is negative, the sender has reacted, and hence there was loss. This method turned out to be problematic, because `cwnd` is gradually reduced down to `ssthresh`

⁴We only predict loss from a “DropTail” (FIFO) queue, no AQM and no ECN. Our only use of ECN is in Section IV, where it is a sender-side vehicle for us to force TCP to back-off when our model predicts packet loss.

during loss recovery. It therefore gave us a model that often predicted loss immediately *after* it happened.

Thus, instead, we labeled the data by comparing the `cwnd` value of the current measurement (the one that should be labeled) with the `cwnd` values of *both* the previous and the next measurement. If the `cwnd` value of the previous measurement was the same or smaller than the current, and the `cwnd` value of the next measurement was smaller, the current measurement was labeled as “Lost”. Otherwise, the current measurement was labeled as “Not lost”.⁵ This increased the imbalance of our dataset, and the labeling requirement of having both a preceding and succeeding `cwnd` value also required us to remove some more of the data in the cleaning step when the `cwnd` field was missing in the `ss` output.

A note about data reliability: Since `ss` is polling based, it can miss datapoints; this makes all of the features that reflect historical information (e.g., `min_rtt`, `cwnd_diff` etc.) inherently unreliable, and it makes our labeling procedure unreliable as well. This is why we included the `data_segments_sent` feature, which will at least give our models a hint about the reliability of such “historical” features in a data sample. As an alternative, we could have used more direct means to access the information from the kernel, e.g. via eBPF [54]. The `ss` method has the advantage of being particularly simple and lightweight, making replication of our results easy.

5) FEATURE IMPORTANCES

After training and tuning two separate models for Reno and Cubic (we will elaborate on this in Subsection III-C), we obtained the feature importances using the `feature_importances_` property of the `XGBClassifier`. These values are shown in Table 1. We can see that the model gave zero importance to the `timer_name` and `retrans` features. This is because `timer_name` turned out to be constant in the `ss` output (always indicating the RTO), and `retrans` probably did not matter for *future* losses.

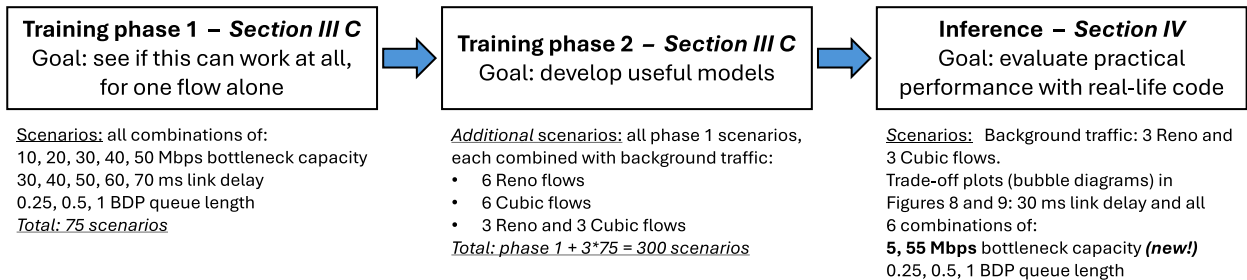
In line with our expectations, all other features had a certain degree of importance for the model; with a few exceptions, these importance values were spread out relatively equally. The most important feature is `expire_time`. Since the RTO timer is refreshed whenever an ACK arrives, this feature will show the most unusual behavior after packet loss, when TCP has reacted. This may make this feature important because, at this point, due to the lower sending rate, another immediate packet loss is particularly unlikely. As we can see in Table 2 and Table 3, most False (“Not lost”) samples that directly follow a True (“Lost”) sample have a significantly larger `expire_time` value.

The `last_send` feature reflects the time since the last packet was sent (different from `data_segments_sent`,

⁵Note that, different from our labels, the `cwnd_diff` feature strictly models the difference from the *preceding* `cwnd` value (else, this feature would look into the future and could not be used for inference).

TABLE 1. Features selected or derived from values in the `ss` output, and their importance. We obtained importance values from the `feature_importances_` property of the `XGBClassifier` after training and tuning models for Reno and Cubic (see Subsection III-C).

Feature	Description	ss field	Importance (Reno)	Importance (Cubic)
<code>timer_name</code>	Name of the timer. We identified the retransmission timeout (RTO) as relevant due to its association with congestion. We therefore used this feature to distinguish this timer from the others.	<code>timer_name</code>	0.00	0.00
<code>expire_time</code>	How long until the timer will expire. This feature is chosen because it is a function of RTT.	<code>expire_time</code>	0.26	0.23
<code>retrans</code>	How many times a retransmission occurred. This is chosen because it could be relevant to determine the degree of congestion.	<code>retrans</code>	0.00	0.00
<code>rto</code>	TCP re-transmission timeout value. It is also a function of the RTT; a higher RTO value indicates a higher RTT value.	<code>rto</code>	0.04	0.05
<code>rtt</code>	Average round-trip time. A large RTT value indicates queue growth and congestion in the network.	<code>rtt</code>	0.04	0.04
<code>rtt_variance</code>	Mean RTT deviation; also a function of RTT.	<code>rttvar</code>	0.02	0.02
<code>min_rtt</code>	Minimum RTT so far.	Derived from <code>rtt</code>	0.03	0.03
<code>max_rtt</code>	Maximum RTT so far.	Derived from <code>rtt</code>	0.06	0.06
<code>cwnd</code>	Congestion window size.	<code>cwnd</code>	0.07	0.06
<code>cwnd_diff</code>	Difference between the current <code>cwnd</code> value and the previous one that was not the same.	Derived from <code>cwnd</code>	0.13	0.15
<code>min_cwnd</code>	Minimum <code>cwnd</code> so far.	Derived from <code>cwnd</code>	0.03	0.03
<code>max_cwnd</code>	Maximum <code>cwnd</code> so far.	Derived from <code>cwnd</code>	0.02	0.03
<code>ssthresh</code>	Slow Start threshold. Chosen because it contains information about the maximum <code>cwnd</code> value that has worked in the past.	<code>ssthresh</code>	0.05	0.05
<code>min_ssthresh</code>	Minimum <code>ssthresh</code> so far.	Derived from <code>ssthresh</code>	0.04	0.04
<code>max_ssthresh</code>	Maximum <code>ssthresh</code> so far.	Derived from <code>ssthresh</code>	0.02	0.02
<code>data_segments_sent</code>	Data segments sent between the current and previous measurement. This could indicate the reliability of some other parameters.	<code>data_segs_out</code>	0.04	0.03
<code>last_send</code>	Time since the last packet was sent. This is chosen because, due to ACK-clocking, there could be a positive correlation between congestion from other traffic and this value.	<code>lastsnd</code>	0.14	0.13
<code>pacing_rate</code>	Pacing rate. This is a function of the RTT, and is chosen because its calculation depends on <code>cwnd/rtt</code> (among other things), which is in principle roughly as important as the <code>cwnd</code> .	<code>pacing_rate</code>	0.02	0.03

**FIGURE 1.** This paper's three steps of model design and evaluation. Data from phase 1 were also used for a preliminary investigation in Subsection IV-A (Figure 3).

this does not relate to the time of taking an `ss` measurement). Due to TCP's ACK clocking, this depends on the rate at which ACKs arrive—which will grow when a queue grows. Thus, it continuously grows as samples get closer to a True case, before dropping down to a substantially smaller value again for the False samples directly following the True case. This makes the feature a congestion indicator of sorts, which probably explains its high importance.

The third feature with a particularly high importance value, `cwnd_diff`, was probably found to be important for the same reason as the others: generally, after packet loss,

`cwnd_diff` will be negative, indicating that the monitored TCP connection has backed off and yet another immediately following packet loss is unlikely.

All of these three most important features share the ability to distinguish between the situation before and immediately after a packet loss, when the chance for another packet loss is particularly small.

C. TRAINING

We approached our design in two phases, followed by interference tests, as shown in Figure 1. At first, we only

TABLE 2. An excerpt of the Reno training dataset, showing a subset of features from Table 1 and the binary “lost” label. The thick lines separate four groups of five consecutive samples from the same TCP test; each group comes from a different TCP test. All groups of five samples include one True sample, the two False samples that occurred right before, and the two False samples that occurred right after.

cwnd	expire_time	rtt	cwnd_diff	ssthresh	data_segments_sent	pacing_rate	lost (label)
262	164	78.795	1	132	12	46.2	False
262	168	78.835	1	132	12	46.2	False
262	168	78.793	1	132	12	46.2	True
180	280	80.494	-6	133	6	60.4	False
174	280	80.503	-6	133	6	61.5	False
495	200	119.45	1	248	16	57.6	False
495	200	119.437	1	248	21	57.6	False
496	248	119.427	1	248	16	57.7	True
488	316	119.598	-8	248	6	57.9	False
298	320	120.294	-7	248	8	79.5	False
115	204	138.074	1	58	2	11.6	False
116	204	138.072	1	58	2	11.7	False
116	20	137.931	1	58	5	11.8	True
114	336	137.957	-2	58	1	11.9	False
77	340	137.286	-3	58	2	15.7	False
18	276	105.988	-2	13	0	4.46	False
18	268	105.988	-2	13	0	4.46	False
18	264	105.988	-2	13	0	4.46	True
15	304	105.669	-3	13	2	4.74	False
15	304	105.615	-3	13	1	4.87	False

TABLE 3. An excerpt of the Cubic training dataset, showing a subset of features from table 1 and the binary “lost” label. The thick lines separate four groups of five consecutive samples from the same TCP test; each group comes from a different TCP test. All groups of five samples include one True sample, the two False samples that occurred right before, and the two False samples that occurred right after.

cwnd	expire_time	rtt	cwnd_diff	ssthresh	data_segments_sent	pacing_rate	lost (label)
300	240	114.847	1	128	9	36.3	False
301	240	115.312	1	128	9	36.3	False
302	240	115.724	1	128	9	36.3	True
299	320	120.028	-3	217	6	38.7	False
295	316	119.985	-4	217	6	39.3	False
112	96	44.623	1	78	6	34.9	False
112	92	44.623	1	78	0	34.9	False
112	88	44.623	1	78	0	34.9	True
78	244	47.288	-4	78	9	33.2	False
78	236	39.652	-4	78	9	48.4	False
7	256	56.066	1	4	0	1.98	False
7	252	56.066	1	4	0	1.98	False
7	248	56.066	1	4	0	1.98	True
6	248	51.91	-1	4	1	2.68	False
4	248	49.041	-2	4	2	2.83	False
111	96	44.74	1	78	10	34.5	False
112	96	44.514	1	78	8	35.0	False
112	96	44.461	1	78	10	35.0	True
81	72	32.289	1	78	8	34.9	False
82	72	32.627	1	78	8	34.9	False

used the datasets containing a single flow with varying link capacities, link delays and queue lengths. The idea was that, despite the widely varying network conditions, the regularity in the behavior of a single sender with no background traffic should make it relatively easy for an ML model to pick out the cases where packet loss should happen. Then, we proceeded to phase two, where we used the complete data including background traffic scenarios.

We created a classifier for Reno and another classifier for Cubic, trained on cases where the “foreground” flow (which we queried with `ss`) was configured to use Reno or Cubic,

respectively. Three datasets for each of these two models were created from the original labeled and cleaned dataset:

- 1) **A training dataset** for training the model by fitting it to the data.
- 2) **A validation dataset** for evaluating the performance of the model when tuning hyperparameters.
- 3) **A final testing dataset** for testing the final model on data that it has not encountered during training or tuning.

To ensure that all sets had a consistent ratio of lost and not lost packets, we used stratified sampling, where we

sampled the measurements based on the “lost” column. This ensured that the proportion of samples for each class (i.e., lost vs. not lost packets) was consistent. We split the data with a 50/25/25 ratio: 50% for training, 25% for validation, and 25% for final testing. The final test set was not used until the end. The seed was fixed for shuffling and sampling. We did this to ensure that we obtained the same split each time, aiding in reproducibility of the results.

We tuned various hyperparameters to improve model performance. The choice of which parameters to tune was based on the relevant section in the XGBoost documentation (“Notes on Parameter Tuning”).⁶ The hyperparameters were tuned manually by supplying various values to the `XGBClassifier` when training, comparing the F1 scores of the resulting classifiers, and choosing the best value based on the score. We chose the F1 score as a metric instead of accuracy because the F1 score is more suitable when the data is heavily imbalanced, as in our case. The tuned hyperparameters are shown with their default and tuned values in Table 4.

Table 5 displays the number of samples for each class in the training, validation, and test sets for the Reno and Cubic datasets in the two phases.

Table 6 shows the precision, recall and F1 score for the Reno and Cubic models in both phases. Overall, these results are not very encouraging, as all of the numbers are low. Low precision means that there is a large rate of false positives, which can be devastating—e.g., in the way we later applied the model, such a misinterpretation would cause a sender to back off when there really is no need to do so. A low recall value means that many instances of loss are missed. The F1 score combines precision and recall into a single number using a harmonic mean, such that, again, a higher number means better model performance.

Going from phase 1 to phase 2, the model has slightly lower data imbalance to cope with. However, we can see that the increased scenario complexity caused it to perform even worse—most importantly, having a precision value of less than 0.5 means that, more often than not, the model gets the result wrong, i.e. there are more false than correct positives. Altogether, judging *only* from table 6, these models are useless, in phase 2 in particular. However, note that, even though the performance is poor, the results between the validation and test set are very similar, indicating that the models generalize well (do not overfit to the training data).

An investigation of our dataset showed us that the low precision can be partially explained by there being very little difference in feature values for the samples labeled as “Lost” and the samples directly preceding them (labeled as “Not Lost”). We can clearly see this in Tables 2 and 3: generally, the differences between the separate groups shown in these tables are much larger than the differences *within* each group. **At this point, we saw three ways to proceed: i) change the way we label data, to assign positive labels to these**

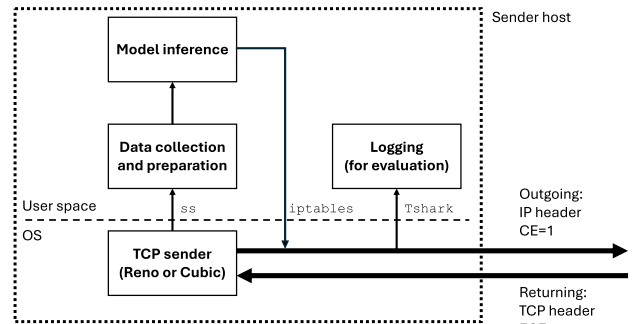


FIGURE 2. Test setup for model inference. Thin arrows represent control flow, thick arrows represent data traffic.

“almost” lost cases too; ii) re-formulate the problem as a regression instead of a binary classification, where we define a notion of being “close to loss”; iii) see how well our current models work in a practical setting. We chose option iii), and report on our findings in the next section.

IV. MODEL INFERENCE AND RESULTS

As already explained, our goal was to test if our model can be beneficially applied in a simple real-life network setting; for this, we opted to let the sender back off via ECN in case the model predicts that a loss would happen (for better visualization, we made the decision to *really* back off or not configurable: some of our diagrams show “normal” TCP behavior with indications where the model *would have* decided to back off).

On the sender side, we started a process for polling data using `ss` and ran `TShark`⁷ in capture mode to monitor both outgoing and incoming packets; the resulting `pcap` files were later used to calculate metrics such as throughput and retransmissions. The `ss` inputs were passed through a data preparation pipeline before carrying out inference; all of this code was written in Python. Data preparation, as determined with the Python `timeit`⁸ module, took about 0.15 ms, and inference took approximately 5 ms. All of our test scenarios had an RTT of at least 30 ms, such that inference was always fast enough to obtain at least one prediction per RTT (which we deemed enough, as TCP only reacts to loss once per RTT). Our code consisted of separate daemons that monitored changes in files on the disk. In a real production system, with optimized code in, e.g., C, it is likely that inference could be fast enough for many practical network scenarios with much shorter RTTs.

We enforced the ECN reaction by marking outgoing data packets as “Congestion Experienced” (CE) at the sender when the model said that this should be done, using a rule of the Linux program `iptables`; as a result, the receiver set the “ECN Echo” (ECE) flag in the TCP header, causing the sender to back off and set the “Congestion Window Reduced” (CWR) flag in the TCP header. For a more

⁷TShark is a terminal version of Wireshark: https://www.wireshark.org/docs/wsug_html_chunked/AppToolstshark.html

⁸<https://docs.python.org/3/library/timeit.html>

⁶https://xgboost.readthedocs.io/en/stable/tutorials/param_tuning.html

TABLE 4. XGBoost hyperparameters for the Reno and Cubic models.

Hyperparameter	Default value	Tuned value (Reno)	Tuned value (Cubic)
Scale pos weight	1	5	5
Learning rate	0.3	0.3	0.3
Gamma	0	9	9
Max depth	6	12	10
Min child weight	1	1	8
Subsample	1	1	1
Colsample bytree	1	1	0.8

TABLE 5. Number of samples for each class in the training, validation, and test sets.

Dataset	Total samples	Samples labeled as lost	Total samples	Samples labeled as lost
Reno	Phase 1		Phase 2	
Training	489464	3294 (0.7%)	7283737	86882 (1.2%)
Validation	241080	1622 (0.7%)	3587513	42792 (1.2%)
Test	243515	1639 (0.7%)	3623750	43224 (1.2%)
Cubic	Phase 1		Phase 2	
Training	433243	3513 (0.8%)	7120069	65805 (0.9%)
Validation	213389	1730 (0.8%)	3506900	32412 (0.9%)
Test	215545	1748 (0.8%)	3542324	32739 (0.9%)

TABLE 6. Precision, recall and F1 score of the Reno and Cubic models.

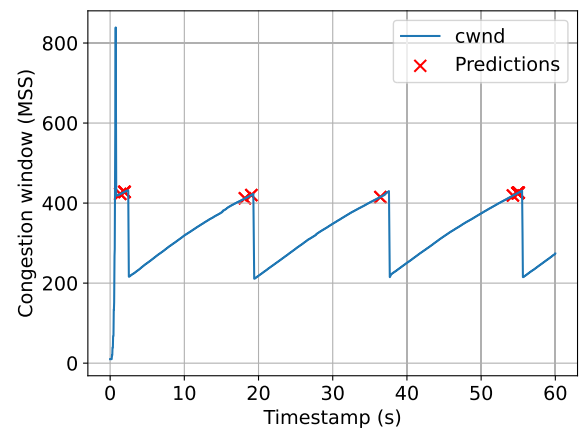
Dataset	Precision	Recall	F1 score	Precision	Recall	F1 score
Reno	Phase 1			Phase 2		
Validation	0.6384	0.1720	0.2710	0.3411	0.4876	0.4014
Test	0.6364	0.1836	0.2850	0.3438	0.4920	0.4048
Cubic	Phase 1			Phase 2		
Validation	0.6055	0.1393	0.2265	0.3233	0.4206	0.3656
Test	0.6450	0.1476	0.2402	0.3203	0.4168	0.3622

immediate sender response, it would also have been possible to directly set ECE in an incoming TCP ACK (this would have saved an RTT), but this would violate the ECN standard (the receiver would not expect to see a CWR response). Also, spending an extra RTT did not matter for our results due to the nature of our predictions (this is clear from the position of the prediction marks in Figures 4 and 6: almost all of them are several RTTs below the peak). Our test setup is shown in Figure 2.

A. EVALUATION

A first investigation of cases without background traffic, using the phase 1 scenarios, showed us that the model can do a very good job at predicting when cwnd should be reduced; an example scenario is depicted in Figure 3. Note that the model prediction loss slightly *before* it really happens—which is, in fact, what we want. Yet, the predictions in Figure 3 are in fact false positives of the binary classification, which has lowered the model’s precision, recall and F1 score in the preceding investigation.

We directly moved on to cases with background traffic (3 Reno and 3 Cubic flows). All following results were obtained with tests using a bottleneck capacity of 5 and 55 Mbit/s—no tests using these values were contained in the training, validation or test sets.

**FIGURE 3.** cwnd and model predictions for TCP Reno with 70 ms delay, 50 Mbps bandwidth, 0.5 BDP queue size, no background traffic, and 0.25 classification threshold.

For each scenario, consisting of a specific combination of delay, bandwidth, and queue size, we ran tests with and without model inference enabled. Since our data are heavily imbalanced towards “Not lost”, with the default classification threshold of 0.5, the model would often decide not to do anything. We therefore applied three different classification thresholds on the probability obtained from XGBoost with the `predict_proba` function: 0.1, 0.25,

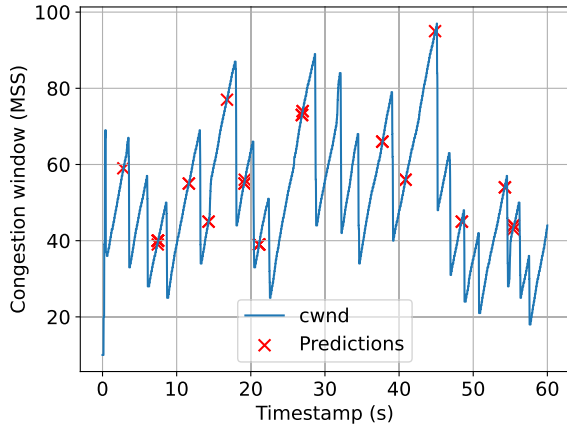


FIGURE 4. cwnd and model predictions for TCP Reno with 70 ms delay, 55 Mbps bandwidth, 0.5 BDP queue size, background traffic, and 0.5 classification threshold.

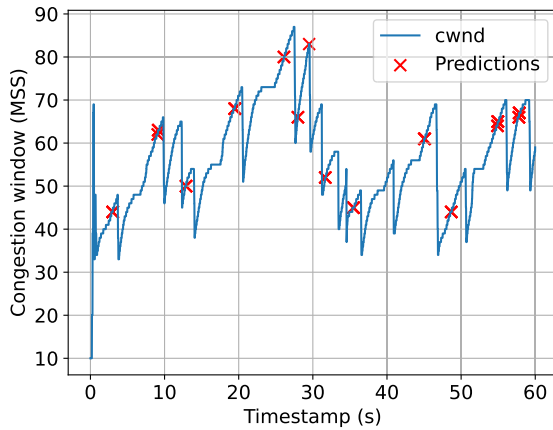


FIGURE 5. cwnd and model predictions for TCP Cubic with 70 ms delay, 55 Mbps bandwidth, 0.5 BDP queue size, background traffic, and 0.5 classification threshold.

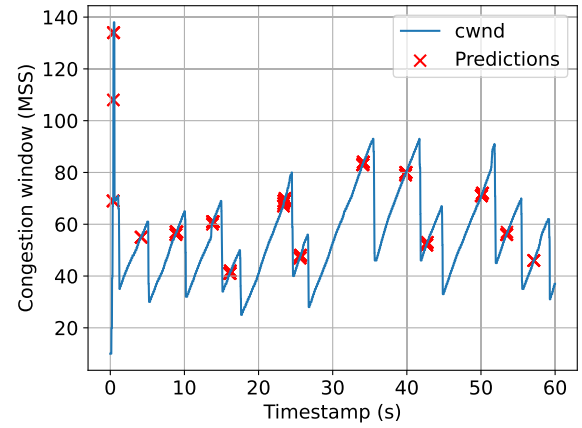


FIGURE 6. cwnd and model predictions for TCP Reno with 70 ms delay, 55 Mbps bandwidth, 1 BDP queue size, background traffic, and 0.5 classification threshold.

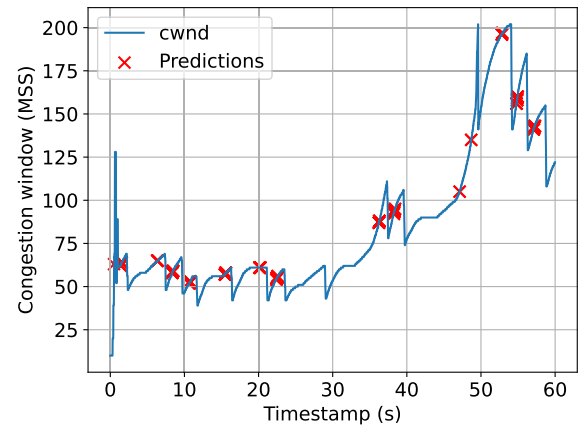


FIGURE 7. cwnd and model predictions for TCP Cubic with 70 ms delay, 55 Mbps bandwidth, 1 BDP queue size, background traffic, and 0.5 classification threshold.

and 0.5. This yields four different groups of results for each scenario:

- 1) **No model inference enabled:** This group serves as a baseline for comparison with the other groups.
- 2) **0.1 threshold:** This group contains the results where model inference was enabled and the classification threshold was configured to 0.1, so that samples above that threshold were classified as “Lost”, and samples below that threshold were classified as “Not Lost”. This tunes the model such that it will decide that loss happens when it sees a chance of at least 10% for it, and will only decide that there is no loss when it is at least 90% certain that there will indeed be no loss.
- 3) **0.25 threshold:** Same as for 0.1 but with a classification threshold of 0.25.
- 4) **0.5 threshold:** Same as for 0.1 and 0.25 but with a classification threshold of 0.5. This is the default choice for most classification tasks, where a choice for “Lost” or “Not Lost” is made when the model considers one of these cases more likely.

For each scenario and each of these groups, the relevant test was run five times, and we tracked the average throughput

and number of retransmissions. To have a meaningful test length, we ensured that, in all our tests, the TCP sender under investigation experienced at least 10 sawteeth in congestion avoidance.

Figures 4 and 6 show the congestion window of a flow configured with Reno and background traffic (3 Reno and 3 Cubic flows) with a queue size of 0.5 and 1 BDP, respectively. The network was configured to use 70 ms RTT and 55 Mbps bandwidth. These are cases from the “no model inference enabled” results group; we just used the model to produce the “X” marks, which indicate where the sender would have backed off (via ECN), in case we had model inference enabled for it. These predictions were produced with the default classification threshold of 0.5. Figures 5 and 7 show the same scenarios for Cubic. We can see the linear, concave and convex increase behavior of the TCP-friendly, convex and concave regions of Cubic, respectively, as described in Section II-A.

As these diagrams illustrate, **most of the predictions are reasonably accurate, even though almost all of them would play out as “bad” cases in the preceding model evaluation since they classify a “loss” slightly before it really happens.** This is more obvious in the 0.5 BDP queue

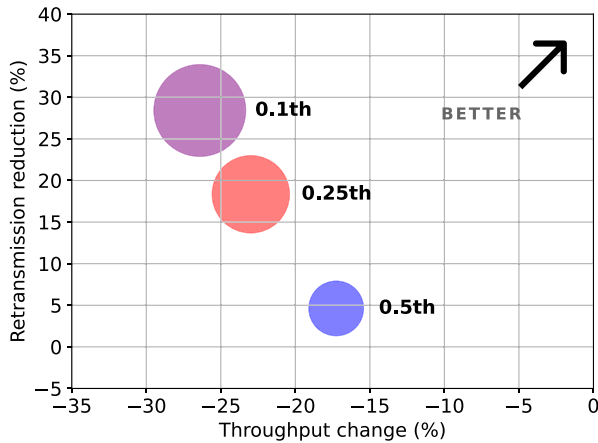


FIGURE 8. Trade-off between retransmission reduction and throughput change for TCP Reno when running with background traffic with model inference enabled at classification thresholds 0.1, 0.25 and 0.5.

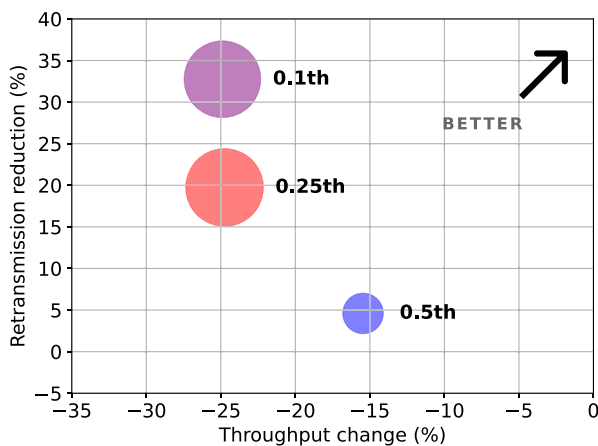


FIGURE 9. Trade-off between retransmission reduction and throughput change for TCP Cubic when running with background traffic with model inference enabled at classification thresholds 0.1, 0.25 and 0.5.

size cases, where the model predictions are generally very close the peaks of each sawtooth (or cubic curve, in case of Cubic), with some exceptions (e.g., at approximately 14 seconds in case of Figure 4, and 28 seconds in Figure 5). At least in case of Reno, predictions tend to look worse with a larger queue size (Figure 6), as they are further below the peak. However, note that a larger queue means a larger range during which the RTT grows, giving the model a wider range in which it is likely to predict that a loss is soon going to happen. Indeed, these marks are where one might expect a proactive congestion control mechanism to react—when the queue begins to grow. The diagrams also show that the model has missed some sawteeth—e.g., the third peak in Figure 4. This is the trade-off that we configure with choosing a threshold. A smaller threshold produces more “X” marks, but they are less likely to all be close to the peaks.

The choice of the classification threshold implements a trade-off between sometimes reacting too early (less retransmissions, less throughput) and sometimes not reacting at all (more retransmissions, more throughput). We illustrate this trade-off in Figures 8 and 9, for which we used a delay

value of 30 ms. We can see that a threshold value of 0.5 offers the most precise behavior (the distribution is the thinnest, i.e. the circle is the smallest in the diagram), generally reducing retransmissions by approximately the same percentage (5%), with a throughput degradation of 16% in case of Cubic and slightly more in case of Reno. The lower classification thresholds 0.1 and 0.25 often caused an overreaction. For both Reno and Cubic, the relationship is nonlinear: lower thresholds reduce retransmissions more significantly than they reduce throughput. This is particularly obvious in the Cubic case, where going from a threshold value of 0.25 to 0.1 reduced retransmissions by around 12% more with only a negligible further throughput degradation.

The choice of threshold could be treated as a parameter where the application could dictate the best value depending on what is more important: fewer retransmissions or higher throughput.

V. CONCLUSION

In this paper, we have investigated the topic of real-time packet loss prediction. Utilizing two XGboost models, which were trained and fine-tuned using data gathered from numerous emulation tests with Reno and Cubic connections over a single bottleneck, we have demonstrated how ECN can be used to make a TCP Reno or Cubic sender back off *before* loss would have happened. Our results confirm that our models can indeed be applied in a practical system in real time, and yield benefits even when using a very simple mechanism such as an artificial ECN signal. All the source code, ML models and datasets are publicly available.⁹

The present work has focused on two separate models, one for Reno and one for Cubic. It is currently unknown if our models would also work for different congestion control mechanisms—and if so, would that require a new model for every new mechanism, or could a more generally applicable model be produced? Such a more general model could have the benefit of being applicable to a wider range of use cases, e.g. by carrying out predictions inside the network on traffic coming from multiple sources.

Feature design and data cleaning could also be improved further. For example, similar to our model, Cubic uses `max_cwnd`, but it refreshes this value whenever loss happened. We could test with a similar logic to update our max/min values (or add them as additional features). In our offline learning approach, the data quality improves as a connection progresses; would it then be better to filter out data from the beginning of a connection?

We have leveraged ECN to reduce the sending rate proactively based on model predictions in order to reduce the number of retransmissions. Instead of reducing the sending rate, as future work, different use cases could be investigated—e.g., to use Forward Erasure Correction (FEC) to dynamically decide when to add redundancy to the data at the sender based on model predictions. If implemented

⁹<https://github.com/safikul/packet-loss-prediction>

effectively, this approach could theoretically lower the overhead commonly linked to FEC implementations. This can be useful because packet loss at the transport layer is relatively rare, resulting in most of the overhead being in vain.

All our datasets were created in virtual environments using Mininet. Such controlled testbed experiments cannot fully capture the variability and complexity of real-world networks. Thus, a particularly important next step is to ensure broader applicability by amending the training data with a much wider range of scenarios, e.g. including different types of background traffic and more heterogeneous network links (cellular, WiFi, ..). As test data, it would then be possible to use connection information gathered from publicly available network traces, such as MAWI¹⁰ or CAIDA datasets,¹¹ in addition to testing the resulting system in real life as opposed to network emulation.

We have shown that using ML to obtain loss predictions that can be beneficially applied in a real-time system is realistic. We hope that this paper can inspire its readers to not only join us as we plan to take the next steps towards a more useful, broadly applicable model, but also find new interesting use cases for its application.

REFERENCES

- [1] E. Blanton, D. V. Paxson, and M. Allman, *TCP Congestion Control*, Standard RFC 5681, Sep. 2009. [Online]. Available: <https://www.rfc-editor.org/info/rfc5681>
- [2] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, *CUBIC for Fast and Long-Distance Networks*, Standard RFC 9438, Aug. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9438>
- [3] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Queue*, vol. 14, no. 5, pp. 20–53, Oct. 2016. [Online]. Available: <http://queue.acm.org/detail.cfm?id=3022184>
- [4] S. Shalunov, G. Hazel, J. Iyengar, and M. Kühlewind, *Low Extra Delay Background Transport (LEDBAT)*, Standard RFC 6817, Dec. 2012. [Online]. Available: <https://www.rfc-editor.org/info/rfc6817>
- [5] Y. Zaki, T. Pötsch, J. Chen, L. Subramanian, and C. Görg, "Adaptive congestion control for unpredictable cellular networks," in *Proc. ACM Conf. Special Interest Group Data Commun.* New York, NY, USA, Aug. 2015, pp. 509–522, doi: [10.1145/2785956.2787498](https://doi.org/10.1145/2785956.2787498).
- [6] V. Arun and H. Balakrishnan, "Copa: Practical delay-based congestion control for the Internet," in *Proc. 15th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, Apr. 2018, pp. 329–342. [Online]. Available: <https://www.usenix.org/conference/nsdi18/presentation/>
- [7] X. Zhu, M. A. Ramalho, and S. M. de la Cruz, *Network-Assisted Dynamic Adaptation (NADA): A Unified Congestion Control Scheme for Real-Time Media*, Standard RFC 8698, Feb. 2020. [Online]. Available: <https://www.rfc-editor.org/info/rfc8698>
- [8] I. Johansson and Z. Sarker, *Self-Clocked Rate Adaptation for Multimedia*, Standard RFC 8298, Dec. 2017. [Online]. Available: <https://www.rfc-editor.org/info/rfc8298>
- [9] S. Floyd, D. K. K. Ramakrishnan, and D. L. Black, *The Addition of Explicit Congestion Notification (ECN) to IP*, Standard RFC 3168, Sep. 2001. [Online]. Available: <https://www.rfc-editor.org/info/rfc3168>
- [10] N. Kuhn, E. Lochin, F. Michel, and M. Welzl, *Forward Erasure Correction (FEC) Coding and Congestion Control in Transport*, Jul. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9265>
- [11] C. A. Grazia, M. Klapez, and M. Casoni, "The new TCP modules on the block: A performance evaluation of TCP pacing and TCP small queues," *IEEE Access*, vol. 9, pp. 129329–129336, 2021.
- [12] R. Adams, "Active queue management: A survey," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 3, pp. 1425–1476, 3rd Quart., 2013.
- [13] B. Briscoe, K. D. Schepper, M. Bagnulo, and G. White, *Low Latency, Low Loss, and Scalable Throughput (LAS) Internet Service: Architecture*, Standard RFC 9330, Jan. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9330>
- [14] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, vol. 11, New York, NY, USA, Aug. 2016, pp. 785–794, doi: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [15] A. Mishra, X. Sun, A. Jain, S. Pande, R. Joshi, and B. Leong, "The great Internet TCP congestion control census," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 3, no. 3, pp. 1–24, Dec. 2019, doi: [10.1145/3366693](https://doi.org/10.1145/3366693).
- [16] M. Hock, R. Bless, and M. Zitterbart, "Experimental evaluation of BBR congestion control," in *Proc. IEEE 25th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2017, pp. 1–10.
- [17] D. Zeynali, E. N. Weyulu, S. Fathalli, B. Chandrasekaran, and A. Feldmann, "BBRv3 in the public Internet: A boon or a bane?" in *Proc. Appl. Netw. Res. Workshop*, vol. 14, New York, NY, USA, Jul. 2024, pp. 97–99, doi: [10.1145/3673422.3674889](https://doi.org/10.1145/3673422.3674889).
- [18] A. Mishra, L. Rastogi, R. Joshi, and B. Leong, "Keeping an eye on congestion control in the wild with nebbi," in *Proc. ACM SIGCOMM Conf.* New York, NY, USA, Aug. 2024, pp. 136–150, doi: [10.1145/3651890.3672223](https://doi.org/10.1145/3651890.3672223).
- [19] N. Cardwell, I. Swett, and J. Beshay, (Jul. 2024). *BBR Congestion Control*. Internet Eng. Task Force. [Online]. Available: <https://datatracker.ietf.org/doc/draft-cardwell-cwgg-bbr/00/>
- [20] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 27, no. 3, pp. 67–82, Jul. 1997, doi: [10.1145/263932.264023](https://doi.org/10.1145/263932.264023).
- [21] A. Gurtov, T. Henderson, S. Floyd, and Y. Nishida, *The New Reno Modification to TCP's Fast Recovery Algorithm*, Standard RFC 6582, Apr. 2012. [Online]. Available: <https://www.rfc-editor.org/info/rfc6582>
- [22] K. Winstein and H. Balakrishnan, "TCP ex machina: Computer-generated congestion control," in *Proc. ACM SIGCOMM Conf. SIGCOMM*, New York, NY, USA, Aug. 2013, pp. 123–134, doi: [10.1145/2486001.2486020](https://doi.org/10.1145/2486001.2486020).
- [23] A. Sivaraman, K. Winstein, P. Thaker, and H. Balakrishnan, "An experimental study of the learnability of congestion control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 479–490, Aug. 2014, doi: [10.1145/2740070.2626324](https://doi.org/10.1145/2740070.2626324).
- [24] W. Wei, H. Gu, and B. Li, "Congestion control: A renaissance with machine learning," *IEEE Netw.*, vol. 35, no. 4, pp. 262–269, Jul. 2021.
- [25] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting congestion control for consistent high performance," in *Proc. 12th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, Oakland, CA, USA, 2015, pp. 395–408. [Online]. Available: <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/dong>
- [26] M. Don, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, "PCC Vivace: Online-learning congestion control," in *Proc. 15th USENIX Symp. Networked Syst. Design Implement.*, Renton, WA, USA, 2018, pp. 343–356. [Online]. Available: <https://www.usenix.org/conference/nsdi18/presentation/dong>
- [27] F. Y. Yan et al., "Pantheon: The training ground for Internet congestion-control research," in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2018, pp. 731–743. [Online]. Available: <https://www.usenix.org/conference/atc18/presentation/yan-francis>
- [28] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on Internet congestion control," in *Proc. 36th Int. Conf. Mach. Learn.*, in Proceedings of Machine Learning Research, vol. 97, K. Chaudhuri and R. Salakhutdinov, Eds., 2019, pp. 3050–3059. [Online]. Available: <https://proceedings.mlr.press/v97/jay19a.html>
- [29] J. Fang, M. Ellis, B. Li, S. Liu, Y. Hosseinkashi, M. Revow, A. Sadovnikov, Z. Liu, P. Cheng, S. Ashok, D. Zhao, R. Cutler, Y. Lu, and J. Gehrke, "Reinforcement learning for bandwidth estimation and congestion control in real-time communications," 2019, *arXiv:1912.02222*.
- [30] V. Sivakumar, O. Delalleau, T. Rocktäschel, A. H. Miller, H. Küttler, N. Nardelli, M. Rabbat, J. Pineau, and S. Riedel, "MVFS-RL: An asynchronous RL framework for congestion control with delayed actions," 2019, *arXiv:1910.04054*.

¹⁰<https://mawi.wide.ad.jp/mawi/>

¹¹https://www.caida.org/catalog/datasets/passive_dataset/

- [31] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the Internet," in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Archit., Protocols Comput. Commun.*, New York, NY, USA, Jul. 2020, pp. 632–647, doi: [10.1145/3387514.3405892](https://doi.org/10.1145/3387514.3405892).
- [32] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTC: Adaptive congestion control with reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 6, no. 3, pp. 445–458, Jul. 2019.
- [33] K. Xiao, S. Mao, and J. K. Tugnait, "TCP-drinc: Smart congestion control based on deep reinforcement learning," *IEEE Access*, vol. 7, pp. 11892–11904, 2019.
- [34] W. Li, H. Zhang, S. Gao, C. Xue, X. Wang, and S. Lu, "SmartCC: A reinforcement learning approach for multipath TCP congestion control in heterogeneous networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 11, pp. 2621–2633, Nov. 2019.
- [35] A. Sacco, M. Flocco, F. Esposito, and G. Marchetto, "Owl: Congestion control with partially invisible networks via reinforcement learning," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.
- [36] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards fair and efficient learning-based congestion control," in *Proc. 19th Eur. Conf. Comput. Syst.*, vol. 4, New York, NY, USA, Apr. 2024, pp. 99–114, doi: [10.1145/3627703.3650069](https://doi.org/10.1145/3627703.3650069).
- [37] L. Roychoudhuri and E. S. Al-Shaer, "Real-time packet loss prediction based on end-to-end delay variation," *IEEE Trans. Netw. Service Manage.*, vol. 2, no. 1, pp. 29–38, Nov. 2005.
- [38] B. A. Arouche Nunes, K. Veenstra, W. Ballenthin, S. Lukin, and K. Obraczka, "A machine learning framework for TCP round-trip time estimation," *EURASIP J. Wireless Commun. Netw.*, vol. 2014, no. 1, pp. 1–22, Dec. 2014.
- [39] A. Giannakou, D. Dwivedi, and S. Peisert, "A machine learning approach for packet loss prediction in science flows," *Future Gener. Comput. Syst.*, vol. 102, pp. 190–197, Jan. 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X19305850>
- [40] A. F. Atiya, S. Goo Yoo, K. To Chong, and H. Kim, "Packet loss rate prediction using the sparse basis prediction model," *IEEE Trans. Neural Netw.*, vol. 18, no. 3, pp. 950–954, May 2007.
- [41] H. R. Mehrvar and M. R. Soleymani, "Packet loss rate prediction using a universal indicator of traffic," in *Proc. IEEE Int. Conf. Communications. Conf. Rec.*, vol. 3, Jul. 2001, pp. 647–653.
- [42] E. Harahap, J. Wijekoon, R. Tennekoon, F. Yamaguchi, S. Ishida, and H. Nishi, "A router-based management system for prediction of network congestion," in *Proc. IEEE 13th Int. Workshop Adv. Motion Control (AMC)*, Mar. 2014, pp. 398–403.
- [43] D. Andreatti, S. Troia, F. Musumeci, S. Giordano, G. Maier, and M. Tornatore, "Network traffic prediction based on diffusion convolutional recurrent neural networks," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2019, pp. 246–251.
- [44] E. Rapaport, I. Poese, P. Zilberman, O. Holschke, and R. Puzis, "Predicting traffic overflows on private peering," 2020, *arXiv:2010.01380*.
- [45] K. Poularakis, Q. Qin, F. Le, S. Kompella, and L. Tassioulas, "Generalizable and interpretable deep learning for network congestion prediction," in *Proc. IEEE 29th Int. Conf. Netw. Protocols (ICNP)*, Nov. 2021, pp. 1–10.
- [46] H. Benadji, L. Zitoune, and V. Vèque, "Predictive modeling of loss ratio for congestion control in IoT networks using deep learning," in *Proc. IEEE Global Commun. Conf.*, Dec. 2023, pp. 2814–2819.
- [47] W. Na, B. Bae, S. Cho, and N. Kim, "DL-TCP: Deep learning-based transmission control protocol for disaster 5G mmWave networks," *IEEE Access*, vol. 7, pp. 145134–145144, 2019.
- [48] L. Diez, A. Fernández, M. Khan, Y. Zaki, and R. Agüero, "Can we exploit machine learning to predict congestion over mmWave 5G channels?" *Appl. Sci.*, vol. 10, no. 18, p. 6164, Sep. 2020. [Online]. Available: <https://www.mdpi.com/2076-3417/10/18/6164>
- [49] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 1–21, 2022.
- [50] S. C. Năhăuș, C. Guyeux, H. H. Arcolezi, R. Couturier, and G. Royer, "A comparison of LSTM and XGBoost for predicting firemen interventions," in *Proc. World Conf. Inf. Syst. Technol.*, Apr. 2020, pp. 424–434.
- [51] M. Zdravković, "XGBoost 'is all you need': the case of forecasting transmitted heat energy in district heating systems," in *Disruptive Information Technologies for a Smart Society*, M. Trajanović, N. Filipović, and M. Zdravković, Eds., Cham, Switzerland: Springer, 2024, pp. 12–22.
- [52] J. Manzoor, I. Drago, and R. Sadre, "The curious case of parallel connections in HTTP/2," in *Proc. 12th Int. Conf. Netw. Service Manage. (CNSM)*, Oct. 2016, pp. 174–180.
- [53] J. Manzoor, I. Drago, and R. Sadre. (2016). *The Curious Case of Parallel Connections in HTTP/2—Presentation Slides*. [Online]. Available: <http://www.cnsm-conf.org/2016/presentations/MiniCNSM2016-Manzoor.pdf>
- [54] O. Tilmans and O. Bonaventure, "COP2: Continuously observing protocol performance," 2019, *arXiv:1902.04280*.



MICHAEL WELZL (Member, IEEE) received the Ph.D. and Habilitation degrees from the University of Darmstadt, Germany, in 2002 and 2007, respectively. He has been a Full Professor with the University of Oslo, Norway, since 2009. He has been active in the IETF and IRTF for many years, e.g., by chairing the Internet Congestion Control Research Group (ICCRG) and leading the effort to form the Transport Services (TAPS) Working Group. He has also participated in several European research projects, including roles, such as the Coordinator and the Technical Manager. His main research interest includes the transport layer.



SAFIQUL ISLAM (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of Oslo, Norway. He is currently an Associate Professor with Oslo Metropolitan University. His research interests include performance analysis, evaluation, and optimization of transport layer protocols. He actively participated in two EU projects (NEAT and RITE) and was the Technical Lead of an International Project ("TCP-in-UDP") with Huawei, China. He is active in the IETF and IRTF, where he has contributed to several IETF/IRTF Working Groups.



MAXIMILIAN VON STEPHANIDES received the M.Sc. degree in programming and system architecture from the University of Oslo, Norway, in 2023. He is currently a Software Engineer with Dintero AS, Oslo, Norway. His research interests include machine learning, transport protocols, and congestion control.

...